SQL Quick Reference and Query Debugging

Filtering and ordering
```
SELECT name, score
FROM students
WHERE score >= 70
ORDER BY score DESC;
```
WHERE filters individual rows before grouping. ORDER BY controls result ordering; without it, row order is not guaranteed.

Aggregates and HAVING
```
SELECT department, COUNT(*) AS total
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```
Use WHERE to filter rows before aggregation and HAVING to filter grouped results. An aggregate such as COUNT(*) generally belongs in HAVING, not WHERE.

NULL handling
```
SELECT name
FROM customers
WHERE email IS NULL;
```
NULL means unknown or missing. Use IS NULL or IS NOT NULL; comparisons like email = NULL do not match NULL values.

Joins
```
SELECT c.name, o.total
FROM customers AS c
JOIN orders AS o ON o.customer_id = c.id;
```
An inner JOIN returns matching rows. A missing or incorrect join condition can multiply rows or produce a Cartesian product.

Parameterized values
```
SELECT id, name
FROM users
WHERE email = ?;
```
Bind values through the database driver’s parameter mechanism. Do not concatenate untrusted input into SQL text; parameters help prevent SQL injection.

Updates and deletes
```
UPDATE inventory
SET quantity = quantity - 1
WHERE item_id = ?;
```
Check the WHERE condition before UPDATE or DELETE. Omitting it can change or remove every row in the table.